

Data Preprocessing II — Placement Guide

This guide covers ten high-impact preprocessing and MLOps topics. The examples use small, leakage-safe patterns that can be adapted to projects.

Feature Engineering

1. Overview

Feature engineering converts raw columns into representations a model can learn from. It is central to tabular ML: fraud, credit risk, forecasting, ranking, and churn models often gain more from good features than from changing algorithms.

2. Intuition

A raw transaction time `2026-07-11 18:30` is hard for a tree or linear model to use directly; `hour=18`, `is_weekend=0`, and `amount/customer_mean` reveal behavior.

3. Prerequisites

Python/Pandas, descriptive statistics, train/test splits, and basic linear/tree models.

4. Core Concepts

Subtopic	Meaning and example	Interview angle
Transformations	<code>log1p(income)</code> reduces right skew.	Why log-transform? Stabilizes scale/outlier influence.
Interactions	<code>price * quantity</code> is revenue.	Linear models need explicit interactions; trees may learn them.
Aggregations	customer’s 30-day mean spend.	Must use only past/available rows.
Missing indicators	<code>age_missing = age.isna()</code> .	Missingness may itself be predictive.

5. Algorithm / Working Process

Define the prediction time, inspect units/distributions, create domain features, fit preprocessing on training data only, transform validation/test, then ablate features to retain useful ones.

6. Mathematical Foundation

Standardization: $z = (x - \mu) / \sigma$. A linear model uses $\hat{y} = w^T x + b$; interaction features extend this with terms such as $w_{ij} x_i x_j$. Feature selection should optimize validation loss, not training loss.

7. Practical Implementation

```
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

def make_features(df):
    x = df.copy()
    x["income_log"] = x["income"].clip(lower=0).pipe(lambda s:
__import__("numpy").log1p(s))
    x["revenue"] = x["price"] * x["quantity"]
    x["age_missing"] = x["age"].isna().astype(int)
    return x

preprocess = ColumnTransformer([
    ("num", Pipeline([("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler())]), ["age", "income_log",
"revenue"]),
    ("cat", OneHotEncoder(handle_unknown="ignore"), ["city"])
])
```

8. Code Explanation

`make_features` is deterministic and receives raw data. `ColumnTransformer` imputes/scales numeric features and one-hot encodes categories inside a pipeline, preventing split leakage.

9. Training / Evaluation

Compare a baseline with and without each feature group using the business metric (AUC, F1, RMSE). Use the same split; large train-validation gaps indicate overfitting or leaky aggregates.

10. Complexity and Cost

Arithmetic features are $O(nd)$. One-hot encoding can be memory-heavy for high cardinality; rolling/group aggregates may dominate data-processing time.

11. Common Use Cases

Credit scoring, demand forecasting, ad CTR, recommender candidates, predictive maintenance.

12. Common Mistakes

Leakage from future aggregates; fitting scalers globally; encoding IDs as meaningful numbers; exploding one-hot features; retaining duplicate/collinear features blindly.

13. Edge Cases / Limitations

Features decay when behavior, schema, or data collection changes. Sparse entities have unreliable aggregate statistics.

14. Variations

Automated feature generation (Featuretools) helps exploratory work; learned embeddings suit high-cardinality categories; domain features remain most important in placements.

15. Related Topics

Feature selection removes unhelpful features; target encoding handles categorical features; feature stores operationalize features; PCA compresses correlated numeric features.

16. Interview Questions

1. **What is leakage?** Using information unavailable at prediction time.
2. **Why pipeline preprocessing?** It fits transforms only on each training fold.
3. **When use log?** Positive, highly skewed variables.
4. **Do trees need scaling?** Usually no; distance/linear models do.
5. **What is an interaction?** A feature whose effect depends on another.
6. **How validate aggregates?** Compute them with historical data only.
7. **Why missing flags?** Missingness can carry signal.
8. **How handle IDs?** Drop them, aggregate them, or encode carefully.
9. **How select features?** Cross-validated ablation/permutation importance.
10. **Why can more features hurt?** Noise and overfitting.

17. Practice Tasks

Build baseline/feature-ablation models; derive date and group features; audit a notebook for leakage; compare one-hot versus frequency encoding; plot feature drift.

18. Project Ideas

Project	Stack/data	Resume value
Churn features	Pandas, sklearn; Telco churn	Demonstrates business/domain features.
Retail demand	Pandas, LightGBM; Store Sales	Demonstrates lag/rolling features.
Fraud features	SQL, sklearn; IEEE-CIS	Demonstrates point-in-time safety.

19. Quick Revision

Create features available at prediction time; validate by ablation; use $z = (x - \mu)/\sigma$ when scale matters. Trap: future-derived features.

20. Final Cheat Sheet

Input/output: raw columns → useful signals. **Steps:** understand time, transform, pipeline, validate. **Metrics:** task metric. **Pros:** often large gains. **Cons:** leakage/drift risk. **Best:** structured business data.

PCA Preprocessing

1. Overview

Principal Component Analysis (PCA) projects correlated numeric features into fewer orthogonal components that retain maximum variance. It is used for compression, visualization, noise reduction, and speeding models.

2. Intuition

Points shaped like a long diagonal ellipse can be described mainly by the ellipse's long direction. PCA rotates axes to that direction, then may discard the short direction.

3. Prerequisites

Vectors/matrices, mean/variance, covariance, eigenvectors, and feature scaling.

4. Core Concepts

Subtopic	Meaning and example	Interview angle
Centering	Subtract each feature mean.	PCA is variance around the mean.
Components	Orthogonal directions of maximum variance.	Components are linear combinations, not original columns.
Explained variance	Fraction retained by first k components.	Choose <code>n_components=0.95</code> cautiously.
Scaling	Standardize different-unit features.	Otherwise large-unit columns dominate.

5. Algorithm / Working Process

Fit scaler on training data; center/scale; compute SVD of training matrix; keep top components; project train/validation/test with the same fitted objects.

6. Mathematical Foundation

For centered X , covariance $C = X^T X / (n - 1)$. PCA chooses unit vector v_1 maximizing $v^T C v$. SVD $X = U \Sigma V^T$: rows of V^T are components and $\sigma_i^2 / \sum_j \sigma_j^2$ is explained-variance ratio.

7. Practical Implementation

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression

model = make_pipeline(StandardScaler(), PCA(n_components=0.95, random_state=0),
                      LogisticRegression(max_iter=1000))
```

```
model.fit(X_train, y_train)
print(model.score(X_test, y_test))
```

8. Code Explanation

The pipeline learns mean, scale, and component directions from `X_train` only. `0.95` retains the smallest number of components explaining at least 95% variance.

9. Training / Evaluation

Tune retained components with cross-validation; compare downstream validation metric and runtime, not explained variance alone. PCA can remove low-variance but predictive signals.

10. Complexity and Cost

Full SVD is roughly $O(nd \min(n, d))$; transformed storage is $O(nk)$. Randomized/incremental PCA helps large matrices.

11. Common Use Cases

Genomics, sensor features, image eigenfaces, visualization (2D), preprocessing for distance-based models.

12. Common Mistakes

Fitting PCA before splitting; skipping scaling for mixed units; applying it to categorical labels; treating components as causally interpretable.

13. Edge Cases / Limitations

It is linear, outlier-sensitive, and variance-based rather than target-aware. Sparse text often prefers TruncatedSVD.

14. Variations

Kernel PCA captures nonlinear structure; Incremental PCA handles batches; SparsePCA yields sparse loadings. Standard PCA is placement-essential.

15. Related Topics

SVD is the computation behind PCA; LDA is supervised reduction; autoencoders learn nonlinear compression; feature selection preserves original columns.

16. Interview Questions

1. **Why standardize?** Unit scale changes variance.
2. **PCA supervised?** No, it ignores labels.
3. **What is a component?** Weighted sum of input features.
4. **Why orthogonal?** Avoid redundant directions.
5. **Choose k?** CV plus explained variance/runtime.

6. **Can PCA leak?** Yes, if fit on all rows.
7. **Does 95% variance ensure accuracy?** No.
8. **SVD relation?** SVD provides principal directions.
9. **Outlier impact?** They can rotate components strongly.
10. **PCA vs feature selection?** PCA transforms; selection retains columns.

17. Practice Tasks

Visualize Iris in 2D; compare classifier with/without PCA; plot scree curve; reconstruct data using k components; test sensitivity to outliers.

18. Project Ideas

Project	Stack/data	Resume value
Sensor compression	sklearn; UCI HAR	Compression versus accuracy trade-off.
Face explorer	PCA, OpenCV; LFW	Eigenfaces and reconstruction.
Gene classifier	sklearn; gene-expression data	High-dimensional preprocessing.

19. Quick Revision

PCA finds max-variance orthogonal directions. Center/scale, fit train only, then project. Main equation: $X_k = XV_k$. Trap: variance is not relevance.

20. Final Cheat Sheet

Input/output: numeric matrix $\rightarrow$ k dense components. **Key hyperparameter:** `n_components`. **Pros:** compact/fast. **Cons:** lost interpretability and signal. **Best:** correlated continuous features.

Target Encoding

1. Overview

Target encoding replaces a category with a smoothed estimate of its target rate, useful for high-cardinality categorical variables such as postcode, product, or publisher.

2. Intuition

If a seller historically converts 12% of visits while the global rate is 5%, encode that seller near 12%; sellers with only one visit should stay nearer 5%.

3. Prerequisites

Categorical data, means/probabilities, train-validation separation, and cross-validation.

4. Core Concepts

Subtopic	Meaning and example	Interview angle
Global prior	Overall target mean μ .	Fallback for unseen categories.
Category mean	Mean target within category.	High variance for rare categories.
Smoothing	Blend category mean with μ .	Controls overfitting.
Out-of-fold encoding	Encode each train row from other folds.	Required to prevent self-leakage.

5. Algorithm / Working Process

On each training fold, calculate category counts/sums on the other folds and map them to the held-out fold. Fit a final mapping on all training data for validation/test inference.

6. Mathematical Foundation

For category c , $TE(c) = (n_c \bar{y}_c + \alpha \mu) / (n_c + \alpha)$. Large α shrinks rare categories toward global mean. For classification, this estimates $P(y = 1 \mid c)$.

7. Practical Implementation

```
import pandas as pd
from sklearn.model_selection import KFold

def oof_target_encode(train, test, col, y, alpha=20, n_splits=5):
    prior = train[y].mean(); out = pd.Series(index=train.index, dtype=float)
    for fit_i, val_i in KFold(n_splits, shuffle=True,
random_state=42).split(train):
        fit, val = train.iloc[fit_i], train.iloc[val_i]
        stats = fit.groupby(col)[y].agg(["sum", "count"])
        mapping = (stats["sum"] + alpha * prior) / (stats["count"] + alpha)
        out.iloc[val_i] = val[col].map(mapping).fillna(prior)
    stats = train.groupby(col)[y].agg(["sum", "count"])
    mapping = (stats["sum"] + alpha * prior) / (stats["count"] + alpha)
    return out, test[col].map(mapping).fillna(prior)
```

8. Code Explanation

Each row's training encoding is built without its own target. Test categories use the final training-only mapping, and unseen categories receive the prior.

9. Training / Evaluation

Use stratified folds for classification where possible. Tune smoothing and compare against one-hot/frequency encoding using validation AUC/log loss; monitor calibration.

10. Complexity and Cost

Groupbys are approximately linear per fold; memory is one numeric column per encoded feature—far smaller than huge one-hot matrices.

11. Common Use Cases

CTR prediction, marketing response, marketplaces, tabular competitions, high-cardinality categorical columns.

12. Common Mistakes

Encoding training rows from their own targets; fitting on validation/test; no smoothing; encoding a near-unique identifier; failing unseen-category fallback.

13. Edge Cases / Limitations

Rare and drifting categories create unstable estimates. For temporal data, folds must respect time, not random KFold.

14. Variations

Leave-one-out encoding is simple but noisy; CatBoost ordered encoding is leakage-resistant; multiclass uses one encoding per class or class probabilities.

15. Related Topics

One-hot is safer for small cardinality; frequency encoding ignores labels; embeddings are learned representations; cross-validation makes target encoding valid.

16. Interview Questions

1. **Why leakage risk?** The encoded value can contain the row's label.
2. **How prevent it?** Out-of-fold mappings.
3. **Why smoothing?** Rare categories overfit.
4. **Unseen category?** Use global prior.
5. **Best for?** High-cardinality categorical variables.
6. **Can encode regression target?** Yes, conditional mean.
7. **Time series rule?** Use past-only mappings.
8. **Why not unique IDs?** They memorize labels.
9. **Alternative?** CatBoost/one-hot/frequency encoding.
10. **What does alpha do?** Strength of shrinkage.

17. Practice Tasks

Implement OOF encoding; vary alpha; compare AUC with one-hot; test unseen categories; reproduce leakage and explain inflated CV.

18. Project Ideas

Project	Stack/data	Resume value
CTR model	Pandas, CatBoost; Criteo	High-cardinality categorical ML.

Project	Stack/data	Resume value
House-price model	sklearn; Ames	Regression encoding workflow.
Retail conversion	SQL/Pandas; online retail	Leakage-safe aggregates/encoding.

19. Quick Revision

Encode categories by smoothed target mean: $(sum + \alpha\mu)/(count + \alpha)$. Use OOF for train and training-only map for test. Trap: label leakage.

20. Final Cheat Sheet

Input/output: category $\rightarrow$ numeric target statistic. **Keys:** folds, smoothing, prior. **Metric:** task AUC/RMSE.

Pros: compact/powerful. **Con:** leakage-prone. **Best:** high-cardinality tabular data.

Time-Based Split

1. Overview

A time-based split trains on the past and evaluates on the future, simulating deployment for forecasting, event prediction, and any data with temporal dependence.

2. Intuition

Predicting next month's sales using future months is like reading tomorrow's newspaper before an exam. Random splitting creates this unrealistically easy setting.

3. Prerequisites

Datetime handling, leakage, temporal ordering, and task metrics.

4. Core Concepts

Subtopic	Meaning and example	Interview angle
Cutoff	Train before 2025-10-01; test after.	Mirrors prediction time.
Gap	Exclude days around cutoff.	Prevents label-window overlap.
Rolling window	Fixed recent history.	Handles concept drift.
Expanding window	All prior history grows.	Uses maximum data.

5. Algorithm / Working Process

Sort by event timestamp, define feature availability and label horizon, choose cutoff/gap, fit transforms on train period, evaluate on later period, then repeat across historical cutoffs.

6. Mathematical Foundation

Forecast target can be y_{t+h} from information $\mathcal{F}_t$. Validity requires every feature satisfy $x_t \in \mathcal{F}_t$, never depend on future observations. MAE is $n^{-1} \sum |y - \hat{y}|$.

7. Practical Implementation

```
df = df.sort_values("event_time")
cutoff = pd.Timestamp("2025-10-01")
gap_end = cutoff + pd.Timedelta(days=7)
train = df[df.event_time < cutoff]
test = df[df.event_time >= gap_end]
# Fit every imputer/encoder/model on train only; score on test.
```

8. Code Explanation

Sorting makes chronology explicit. The seven-day gap is optional but protects targets/features whose windows overlap the cutoff.

9. Training / Evaluation

Use multiple backtests. Report average and worst-period metric, plus drift by month/segment. Do not tune repeatedly on the final future test window.

10. Complexity and Cost

One split is cheap; rolling backtests multiply training cost by number of folds. Data sorting is $O(n \log n)$.

11. Common Use Cases

Demand and price forecasting, churn, fraud, delayed labels, click prediction, monitoring production models.

12. Common Mistakes

Random split; leaking future rolling means; joining later snapshots; ignoring label delay; mixing event time with ingestion time.

13. Edge Cases / Limitations

Seasonality can make one cutoff unrepresentative. New products/users may have no history. A severe regime shift limits historical relevance.

14. Variations

Holdout split is simplest; walk-forward validation is standard; purged CV adds an embargo for overlapping financial labels—important for time-series interviews.

15. Related Topics

`TimeSeriesSplit` automates folds; feature engineering needs point-in-time joins; data versioning preserves historical snapshots; drift monitoring detects change.

16. Interview Questions

1. **Why not random split?** It leaks temporal patterns/future state.
2. **What is a gap?** Excluded boundary period.
3. **Expanding vs rolling?** All history versus fixed recent window.
4. **What is backtesting?** Repeating past-to-future evaluations.
5. **Feature availability?** Must exist at prediction timestamp.
6. **What is label delay?** Outcome arrives later.
7. **Use stratification?** Usually chronology is more important.
8. **How detect drift?** Compare distributions/metrics by time.
9. **Final test use?** Once after decisions.
10. **Why embargo?** Prevent overlapping labels leaking.

17. Practice Tasks

Make weekly backtests; compare random versus chronological metrics; add a lag safely; implement expanding windows; audit timestamps in a dataset.

18. Project Ideas

Project	Stack/data	Resume value
Store forecast	sklearn; M5/Store Sales	Proper backtesting.
Churn predictor	Pandas; Telco events	Point-in-time feature design.
Energy demand	PyTorch/sklearn; UCI energy	Temporal drift analysis.

19. Quick Revision

Past trains, future tests; features must be known at time t . Use backtests and gaps for overlapping windows. Trap: random split.

20. Final Cheat Sheet

Input/output: timestamped rows $\rightarrow$ realistic future score. **Steps:** sort, cutoff, gap, train-only fit, backtest.

Metrics: MAE/RMSE/AUC. **Pros:** honest evaluation. **Con:** less train data per fold.

Cross-Validation Split

1. Overview

Cross-validation (CV) repeatedly splits training data to estimate generalization and choose models/hyperparameters more reliably than one validation split.

2. Intuition

Instead of judging a student from one question paper, use several papers and average the scores. Each fold gets a turn as validation.

3. Prerequisites

Train/test separation, metrics, sampling, class imbalance, and pipelines.

4. Core Concepts

Subtopic	Meaning and example	Interview angle
K-fold	K partitions; train K-1, validate 1.	Score mean and spread.
StratifiedKFold	Preserves class proportions.	Default for classification.
GroupKFold	A group never spans train/validation.	Prevents patient/user leakage.
Nested CV	Inner tuning, outer estimation.	Reduces selection bias.

5. Algorithm / Working Process

Hold out a final test set. For each fold, fit every preprocessing/model step on the fold's training partition, score the validation partition, average scores, choose configuration, retrain on all development data, test once.

6. Mathematical Foundation

CV estimate $\hat{R}_{CV} = K^{-1} \sum_{k=1}^K L(y^{(k)}, f_{-k}(X^{(k)}))$. Its standard deviation communicates sensitivity to sampling; it is not an independent final-test guarantee.

7. Practical Implementation

```
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = make_pipeline(SimpleImputer(), StandardScaler(),
                     LogisticRegression(max_iter=1000))
cv = StratifiedKFold(5, shuffle=True, random_state=42)
scores = cross_val_score(pipe, X, y, cv=cv, scoring="roc_auc")
print(scores.mean(), scores.std())
```

8. Code Explanation

The pipeline is re-fit inside each fold by `cross_val_score`; `StratifiedKFold` preserves positive-class rate; `roc_auc` is appropriate when ranking imbalanced binary cases.

9. Training / Evaluation

Use 5 or 10 folds; use repeated CV only when variance matters and cost permits. Tune with `GridSearchCV`/`RandomizedSearchCV`; preserve a final untouched test set.

10. Complexity and Cost

CV trains K models: about K times training time. Parallelize folds if memory permits; each worker holds data/model copies.

11. Common Use Cases

Small/medium tabular datasets, algorithm comparison, hyperparameter tuning, robust benchmark reporting.

12. Common Mistakes

Preprocessing outside the pipeline; standard KFold for grouped data; random folds for time series; scoring accuracy on imbalanced data; reporting best fold only.

13. Edge Cases / Limitations

Very large datasets rarely need expensive CV; correlated/duplicate data invalidates random folds; CV can still overfit after many tuning iterations.

14. Variations

Leave-one-out is high-cost/high-variance; repeated K-fold stabilizes estimates; stratified group CV combines constraints; time-series CV preserves order.

15. Related Topics

Bootstrapping also estimates uncertainty; train/validation/test is simpler; time-based split replaces random CV for temporal data; pipelines prevent leakage.

16. Interview Questions

1. **Why CV?** More reliable validation estimate.
2. **Why final test?** Tuning biases CV estimate.
3. **Why stratify?** Keeps class balance per fold.
4. **When GroupKFold?** Repeated users/patients/devices.
5. **Can scaling leak?** Yes, fit it per fold.
6. **K=5 or 10?** Common compute/bias trade-off.
7. **Nested CV?** Outer performance, inner tuning.
8. **CV for time series?** Use chronological folds.
9. **Mean and std?** Performance and stability.
10. **Why not LOO?** Expensive and often high variance.

17. Practice Tasks

Compare KFold/Stratified/GroupKFold; intentionally leak scaling; tune C with nested CV; plot fold scores; create group-aware patient split.

18. Project Ideas

Project	Stack/data	Resume value
Disease classifier	sklearn; UCI data	Stratified metrics and CV.
Patient risk	sklearn; MIMIC-style groups	Group leakage prevention.
Model benchmark	sklearn; OpenML	Reproducible comparison.

19. Quick Revision

CV averages validation losses across K train/validation rotations. Use matching splitter and pipeline. Trap: choosing splitter by convenience, not data dependence.

20. Final Cheat Sheet

Input/output: development data → validation estimate. **Key:** K, splitter, metric. **Cost:** K fits. **Pros:** robust use of limited data. **Con:** expensive and leakable. **Best:** non-temporal data.

Text Preprocessing

1. Overview

Text preprocessing converts documents into clean, model-compatible representations. The right amount depends on model: sparse models need tokenization/vectorization; transformer models usually need only their native tokenizer.

2. Intuition

"Loved this movie!" becomes tokens and numbers. For TF-IDF, distinctive words get high weight; for BERT, subword tokens preserve context for the model.

3. Prerequisites

Strings/regex, probability, train/test split, vectorization, embeddings, and NLP task metrics.

4. Core Concepts

Subtopic	Meaning and example	Interview angle
Normalization	Unicode/whitespace/case handling.	Do not remove signal blindly.
Tokenization	Text → words/subwords.	Transformers require model-specific tokenizers.
Stopwords/lemmatization	Remove common words / normalize forms.	Often helpful for classical ML, not needed for BERT.
TF-IDF	Importance by document frequency.	Strong baseline for classification.

5. Algorithm / Working Process

Define task and language, inspect noise and labels, split before fitting vocabulary, clean minimally, tokenize/vectorize training text, train model, apply same transform in inference, monitor unknown/length behavior.

6. Mathematical Foundation

TF-IDF: $tfidf(t, d) = tf(t, d) \log((N + 1)/(df(t) + 1)) + 1$. Cosine similarity is $a \cdot b / (||a|| ||b||)$. Tokenizers map text to IDs; transformers learn contextual embeddings from those IDs.

7. Practical Implementation

```
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

text_model = make_pipeline(
    TfidfVectorizer(lowercase=True, ngram_range=(1, 2), min_df=2,
                    strip_accents="unicode"),
    LogisticRegression(max_iter=1000)
)
text_model.fit(train_text, y_train)
pred = text_model.predict(test_text)
```

8. Code Explanation

The vectorizer learns its vocabulary and IDF from training text only. Word bigrams preserve phrases like “not good”; logistic regression is a fast, interpretable baseline.

9. Training / Evaluation

Use stratified split for classification. Report macro-F1 for imbalanced/multiclass labels, inspect confusion matrix, and check errors involving negation, spelling, and long documents.

10. Complexity and Cost

Sparse TF-IDF cost is roughly proportional to tokens; vocabulary and n-grams increase memory. Transformer training/inference costs grow roughly quadratically with sequence length.

11. Common Use Cases

Sentiment, spam, ticket routing, search, document classification, RAG chunk preparation.

12. Common Mistakes

Removing negations; fitting vocabulary on test text; using English stopwords for another language; truncating without checking; manually tokenizing before a transformer tokenizer.

13. Edge Cases / Limitations

Sarcasm/context and multilingual code-switching are difficult. PII must be removed or protected. Aggressive cleaning can destroy entities, casing, URLs, and emojis.

14. Variations

Character n-grams handle typos; stemming is cheaper but rougher than lemmatization; sentence embeddings serve similarity/RAG; subword transformers are key for modern NLP.

15. Related Topics

Embeddings represent semantic text; RAG adds chunking/retrieval; tokenization affects LLM cost; target encoding can handle categorical metadata alongside text.

16. Interview Questions

1. **TF-IDF idea?** Frequent in document, rare across corpus.
2. **Why n-grams?** Capture phrases/order.
3. **Stemming vs lemmatization?** Crude truncation versus linguistic base form.
4. **Clean before BERT?** Minimal cleaning; use its tokenizer.
5. **Why train-only vocabulary?** Prevent evaluation leakage.
6. **OOV handling?** Unknown token or subword tokenization.
7. **Macro-F1?** Equal class weighting.
8. **Character n-grams?** Robust to typos/morphology.
9. **Why preserve negation?** Changes sentiment meaning.
10. **RAG preprocessing?** Clean, chunk, embed, index with metadata.

17. Practice Tasks

Build spam TF-IDF baseline; compare word/character n-grams; inspect top coefficients; measure effect of stopwords; chunk a PDF corpus for retrieval.

18. Project Ideas

Project	Stack/data	Resume value
Support router	sklearn; CLINC150	Production-style text classification.
Spam filter	sklearn; SMS Spam	Precision/recall trade-offs.
Mini RAG	sentence-transformers; docs	Chunking and retrieval evaluation.

19. Quick Revision

Text → tokens → vectors → model. TF-IDF rewards discriminative terms; transformers own tokenization. Trap: destructive cleaning or vocabulary leakage.

20. Final Cheat Sheet

Input/output: documents → sparse vectors/token IDs. **Keys:** tokenizer, vocabulary, max length. **Metrics:** macro-F1, precision/recall. **Pros:** enables NLP. **Cons:** language/noise sensitive. **Best:** classify, search, retrieve.

Image Preprocessing

1. Overview

Image preprocessing standardizes pixels and labels so vision models see consistent inputs. It includes decoding, resizing, normalization, augmentation, and task-specific annotation handling.

2. Intuition

Photos vary in camera, brightness, and size. Resize gives the network a consistent canvas; augmentation shows plausible alternate views so it learns the object, not one exact photo.

3. Prerequisites

Arrays/tensors, RGB channels, CNNs or vision transformers, interpolation, and classification/detection metrics.

4. Core Concepts

Subtopic	Meaning and example	Interview angle
Resize/crop	Make 224×224 inputs.	Preserve aspect ratio; avoid distortion.
Normalization	Scale pixels and subtract channel stats.	Match pretrained model's expected stats.
Augmentation	Random flip/crop/color jitter.	Must preserve label semantics.
Label transforms	Transform boxes/masks with image.	Critical for detection/segmentation.

5. Algorithm / Working Process

Verify files/labels and color order, split by subject/source when needed, use training-only stochastic augmentation, use deterministic validation transforms, batch tensors, then normalize exactly as expected by pretrained backbone.

6. Mathematical Foundation

Pixel normalization is $x' = (x/255 - \mu)/\sigma$ per channel. A horizontal flip maps x-coordinate to $W - x$; for boxes it must swap left/right boundaries. Augmentations approximate invariance to valid transforms.

7. Practical Implementation

```
from torchvision import transforms

train_tf = transforms.Compose([
    transforms.Resize(256), transforms.RandomResizedCrop(224),
    transforms.RandomHorizontalFlip(), transforms.ColorJitter(0.15, 0.15),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])
eval_tf = transforms.Compose([
```

```
transforms.Resize(256), transforms.CenterCrop(224), transforms.ToTensor(),
transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])
```

8. Code Explanation

Training transform randomizes only label-preserving appearance/geometry. Evaluation is deterministic. ImageNet normalization matches many pretrained torchvision weights.

9. Training / Evaluation

Split by patient/product/video—not only by image—to avoid near-duplicates across splits. Use accuracy/F1 for classification, mAP for detection, IoU/Dice for segmentation; visually inspect augmented samples.

10. Complexity and Cost

Raw images use $H \times W \times C$ memory; larger resolution increases CNN cost roughly with pixels and ViT attention strongly with patch count. GPU is useful for training; CPU preprocessing can bottleneck.

11. Common Use Cases

Medical imaging, OCR, manufacturing inspection, autonomous driving, retail catalog classification.

12. Common Mistakes

RGB/BGR mix-up; wrong pretrained normalization; augmenting validation/test; applying flips to directional labels; transforming images but not boxes/masks; duplicate leakage.

13. Edge Cases / Limitations

Augmentation can produce unrealistic medical/industrial samples. Compression, corrupt files, extreme aspect ratios, and domain shift require dedicated checks.

14. Variations

Random erasing/MixUp/CutMix regularize classification; histogram equalization can help controlled imaging; test-time augmentation is optional. Basic resize/normalize/flip is placement-essential.

15. Related Topics

Transfer learning depends on normalization; CV split prevents subject leakage; synthetic images augment scarce data; data versioning records labels and transform versions.

16. Interview Questions

1. **Why normalize?** Stable optimization and pretrained compatibility.
2. **Why separate transforms?** Evaluation must be repeatable.
3. **Can flip always help?** No—text, left/right anatomy, driving signs.
4. **Crop risk?** Can remove object/label evidence.

5. **RGB vs BGR?** Channel ordering differs by library.
6. **Detection augmentation rule?** Update boxes/masks too.
7. **What is leakage?** Same subject/near duplicate in both splits.
8. **ImageNet stats always?** Only when matching a model trained with them.
9. **Metric for segmentation?** IoU/Dice.
10. **Why visual QA?** Catches label-breaking transforms.

17. Practice Tasks

Display augmented batches; compare crop policies; find duplicates with hashes; train with/without flips; transform bounding boxes correctly.

18. Project Ideas

Project	Stack/data	Resume value
Waste classifier	PyTorch; TrashNet	Transfer learning pipeline.
Defect detector	OpenCV/PyTorch; MVTec AD	Industrial visual QA.
Skin lesion classifier	torchvision; HAM10000	Patient-aware splitting.

19. Quick Revision

Decode → resize/crop → tensor → normalize; augment train only; transform labels consistently. Trap: source/subject leakage.

20. Final Cheat Sheet

Input/output: image/annotation → normalized tensor/label. **Keys:** resolution, mean/std, augmentation.

Metrics: accuracy/mAP/IoU. **Pros:** robust inputs. **Cons:** semantic transform risk. **Best:** all vision pipelines.

Data Versioning

1. Overview

Data versioning records immutable or reproducible dataset states, metadata, and lineage so an experiment can be repeated, audited, and rolled back. It is MLOps' counterpart to Git for code.

2. Intuition

"model-v4 used `train.csv`" is insufficient if that file changes. A data version is a receipt: exact files, schema, hashes, query, labels, and split definition.

3. Prerequisites

Git concepts, files/object storage, hashes, schemas, experiment tracking, and reproducibility.

4. Core Concepts

Subtopic	Meaning and example	Interview angle
Snapshot	Frozen data state identified by hash/tag.	Supports reproduction/rollback.
Lineage	Source → cleaned set → features → model.	Enables impact analysis.
Metadata	Schema, owner, timestamp, consent, stats.	Data is not just a CSV.
Immutable raw data	Never overwrite source records.	Enables reprocessing.

5. Algorithm / Working Process

Ingest raw data to immutable storage, validate schema/quality, create a version manifest with hashes and code/config references, generate versioned curated data, link model/metrics to that version, and retain according to policy.

6. Mathematical Foundation

Cryptographic hashes map bytes to identifiers: $id = H(file)$. They detect changes, not semantic equivalence. Dataset drift can be measured by distributions, e.g., $PSI = \sum (p_i - q_i) \ln(p_i/q_i)$.

7. Practical Implementation

```
from pathlib import Path
import hashlib, json

path = Path("data/train.parquet")
digest = hashlib.sha256(path.read_bytes()).hexdigest()
manifest = {"dataset": "customer-churn", "version": "2026-07-11",
            "file": str(path), "sha256": digest, "split_seed": 42,
            "schema": {"target": "churn", "rows": 10000}}
Path("data/manifest.json").write_text(json.dumps(manifest, indent=2))
```

8. Code Explanation

The manifest stores a content hash and critical reproduction metadata. In production, store data in versioned object storage and commit the small manifest/config to Git; tools such as DVC can automate this.

9. Training / Evaluation

Log dataset version, feature code version, split seed, model parameters, environment, metrics, and artifacts together. Re-run a prior experiment from its manifest before promoting a model.

10. Complexity and Cost

Hashing is $O(file\ size)$; full snapshots can be storage-expensive. Content-addressed storage/deduplication reduces repeated-file cost.

11. Common Use Cases

Regulated ML, retraining pipelines, labeling projects, A/B investigations, incident rollback, reproducible research.

12. Common Mistakes

Versioning code but not data; mutable “latest” paths; missing split seed; storing secrets/PII in manifests; no schema/quality checks; deleting raw provenance.

13. Edge Cases / Limitations

Live streams and mutable databases need snapshot/query-time semantics. Privacy retention rules may prevent indefinite immutable copies.

14. Variations

DVC tracks files with remote storage; lakehouse time travel versions tables; Delta/Iceberg/Hudi add transactional table history. Concepts matter more than tool choice in interviews.

15. Related Topics

Experiment tracking links metrics to datasets; feature stores version feature definitions; model registries version models; data contracts enforce schemas.

16. Interview Questions

1. **Why version data?** Reproduce and audit models.
2. **Git enough?** Usually not for large binary data.
3. **What to record?** Source, hash, schema, split, transformations.
4. **Snapshot vs lineage?** State versus how it was produced.
5. **Why immutable raw?** Reprocessing/forensics.
6. **How identify a version?** Hash/tag/commit.
7. **What is schema drift?** Column/type/meaning change.
8. **How rollback?** Rebuild using prior manifest/version.
9. **What about PII?** Access controls and retention/deletion policy.
10. **DVC role?** Git-like data pointers plus remote artifacts.

17. Practice Tasks

Create manifests; reproduce a model from a fixed data hash; deliberately change a column; compare DVC and Git-LFS; draw data-to-model lineage.

18. Project Ideas

Project	Stack/data	Resume value
Reproducible churn ML	DVC, MLflow; Telco	End-to-end MLOps evidence.
Labeling audit trail	Parquet, Git/DVC; images	Dataset governance.
Drift-aware retraining	Great Expectations, MLflow	Operational data quality.

19. Quick Revision

Version data states and lineage, not just code. Hashes detect byte changes; manifests make experiments reproducible. Trap: `latest.csv` without a snapshot.

20. Final Cheat Sheet

Input/output: raw/curated data → immutable version + manifest. **Keys:** hash, schema, lineage, split. **Metric:** reproducibility/quality checks. **Pros:** auditability. **Con:** storage/process overhead. **Best:** any serious ML pipeline.

Feature Stores

1. Overview

A feature store is a system for defining, computing, discovering, serving, and monitoring reusable ML features consistently for offline training and online inference.

2. Intuition

Without a feature store, training calculates “customer 30-day spend” in SQL while serving reimplements it in Python—often differently. The store gives both paths one definition.

3. Prerequisites

Feature engineering, SQL/batch processing, timestamps, key-value access, APIs, and MLOps basics.

4. Core Concepts

Subtopic	Meaning and example	Interview angle
Entity	Join key: <code>customer_id</code> .	Feature values belong to entities.
Feature view	Named schema/transformation group.	Reusable contract.
Offline store	Historical data for training.	Supports point-in-time joins.
Online store	Low-latency latest feature lookup.	Serves inference.

5. Algorithm / Working Process

Define entities and feature ownership, ingest batch/stream features with event timestamps, validate and materialize them, create point-in-time-correct training sets from offline history, serve latest approved values online, monitor freshness and skew.

6. Mathematical Foundation

For prediction at time t , a training join must select $f_e(\tau)$ where $\tau \leq t$; this is an as-of join. Training-serving skew is $f_{train} \neq f_{serve}$, a system consistency problem rather than a model loss.

7. Practical Implementation

```
# Minimal feature contract; Feast-style stores formalize this in a registry.
FEATURES = {
    "customer": {"entity_key": "customer_id",
                 "fields": ["spend_30d", "orders_90d", "days_since_last_order"],
                 "event_timestamp": "event_time"}
}

def point_in_time_join(labels, history):
    labels = labels.sort_values("prediction_time")
    history = history.sort_values("event_time")
    return pd.merge_asof(labels, history, left_on="prediction_time",
                          right_on="event_time", by="customer_id",
                          direction="backward")
```

8. Code Explanation

The contract documents entity, fields, and event time. `merge_asof(... direction="backward")` prevents future feature values from joining to past labels.

9. Training / Evaluation

Validate freshness, null rates, ranges, and offline/online parity. Train from point-in-time datasets; at serving log feature values/version and monitor skew, latency, and prediction quality.

10. Complexity and Cost

Offline joins can be large distributed jobs; online reads target millisecond latency and cost memory/storage. Materialization adds pipeline and operational cost.

11. Common Use Cases

Fraud scoring, recommendations, personalization, dynamic pricing, real-time churn risk, shared enterprise features.

12. Common Mistakes

Calling a feature table a feature store; no event timestamps; future leakage in historical joins; separate training/serving definitions; no freshness ownership; online-serving stale data.

13. Edge Cases / Limitations

Not worthwhile for one offline notebook/model. Late events, backfills, deletions, and entity merges require explicit policy.

14. Variations

Offline-only feature registry for batch ML; streaming feature stores for real time; managed platforms (SageMaker, Vertex) versus open source Feast. Understanding offline/online parity is placement-important.

15. Related Topics

Data versioning versions datasets; feature engineering creates features; time split validates as-of correctness; model registry deploys consumers of served features.

16. Interview Questions

1. **What problem does it solve?** Reuse and train-serving consistency.
2. **Offline vs online?** Historical training versus low-latency serving.
3. **What is point-in-time join?** Use only features known then.
4. **Why entity key?** Identifies feature subject.
5. **Feature freshness?** Age since last update.
6. **What is skew?** Training and serving feature mismatch.
7. **Need a store always?** No, overhead may exceed benefit.
8. **How prevent leakage?** Event timestamps/as-of joins.
9. **Materialization?** Move computed values into serving store.
10. **How monitor?** Nulls, ranges, freshness, distributions, latency.

17. Practice Tasks

Implement point-in-time join; make a feature contract; calculate freshness; simulate stale online values; compare offline and online feature samples.

18. Project Ideas

Project	Stack/data	Resume value
Real-time fraud features	Feast, Redis, Parquet	Offline/online parity.
Recommender features	Spark/Feast; MovieLens	Entity and feature views.
Churn platform	FastAPI, Postgres	Feature freshness/API serving.

19. Quick Revision

Feature store = governed reusable features for training and serving. Core rule: as-of joins. Trap: future data and duplicated feature logic.

20. Final Cheat Sheet

Input/output: entity events → offline training set/online feature vector. **Keys:** entity, timestamp, feature view, freshness. **Pros:** reuse/parity. **Con:** operational overhead. **Best:** multiple or real-time models.

Synthetic Data Generation

1. Overview

Synthetic data is artificially generated data designed to mimic useful statistical or semantic properties of real data. It augments scarce data, supports testing, reduces privacy exposure, and enables simulation.

2. Intuition

If you have few defect photos, create plausible varied defect images to train a robust detector—but synthetic images must not replace realistic evaluation data.

3. Prerequisites

Probability/distributions, train-validation separation, privacy basics, generative models, and domain validation.

4. Core Concepts

Subtopic	Meaning and example	Interview angle
Rule-based	Sample age/income within constraints.	Best for tests/simulations.
Statistical	Preserve correlations with copulas.	Compare distributions and relations.
Generative	GAN/VAE/diffusion/LLM creates samples.	Realism ≠ privacy or utility.
Augmentation	Label-preserving variations of existing data.	Different from fully synthetic data.

5. Algorithm / Working Process

Set objective and privacy constraints, profile real training data, choose generator, generate only from development data, enforce schema/business rules, evaluate fidelity/utility/privacy, then train with real+synthetic data and evaluate solely on a held-out real test set.

6. Mathematical Foundation

A generator seeks $p_g(x) \approx p_{data}(x)$. GAN objective: $\min_G \max_D E_{x \sim p_{data}} \log D(x) + E_z \log(1 - D(G(z)))$. Utility is downstream performance on real held-out data; distribution similarity alone is insufficient.

7. Practical Implementation

```
import numpy as np
import pandas as pd

def synthetic_customers(n, seed=42):
    rng = np.random.default_rng(seed)
    age = rng.integers(18, 81, n)
    income = np.exp(rng.normal(10.7 + 0.01 * (age - 40), 0.45, n)).round(2)
    orders = rng.poisson(np.clip(income / 20_000, 0.2, 8))
    return pd.DataFrame({"age": age, "income": income, "orders": orders})

syn = synthetic_customers(1_000)
assert syn.age.between(18, 80).all() and (syn.income > 0).all()
```

8. Code Explanation

This constraint-aware simulator samples plausible—not private or production-faithful—customers. The assertions are quality rules; real systems should add schema validation and compare against real development data.

9. Training / Evaluation

Never report evaluation only on synthetic data. Compare feature distributions, correlations, rare-class coverage, and downstream model metric on untouched real test data; perform privacy/memorization checks for sensitive data.

10. Complexity and Cost

Rule-based generation is cheap. GAN/diffusion training can require substantial GPU compute and curated data; high-resolution generation is expensive. Storage grows with generated volume.

11. Common Use Cases

Test data, simulation, class imbalance augmentation, privacy-conscious prototyping, rare-event and vision-data augmentation.

12. Common Mistakes

Training and evaluating on synthetic data; generating from test data; claiming privacy without attack testing; ignoring rare modes; invalid labels/constraints; synthetic artifacts becoming shortcuts.

13. Edge Cases / Limitations

Generators can amplify bias, miss rare events, memorize records, or produce visually convincing but physically impossible samples. Synthetic data cannot replace real deployment distribution.

14. Variations

SMOTE interpolates minority tabular points; CTGAN models mixed tabular data; VAEs/GANs/diffusion generate complex data; LLMs produce text. SMOTE and evaluation caveats are placement-important.

15. Related Topics

Image augmentation is controlled synthetic variation; differential privacy adds formal protection; data versioning tracks generator/config; feature engineering validates constraints and utility.

16. Interview Questions

1. **Why synthetic data?** Scarcity, testing, augmentation, privacy support.
2. **Does it guarantee privacy?** No; test memorization/disclosure risk.
3. **How evaluate utility?** Train with it, test on real holdout.
4. **What is SMOTE?** Minority interpolation in feature space.
5. **SMOTE limitation?** Can create class overlap/noise.
6. **GAN objective?** Generator fools discriminator.

7. **What is mode collapse?** Generator produces limited variety.
8. **Can synth data fix bias?** Not automatically; it can amplify it.
9. **Why constraints?** Plausibility/validity.
10. **Augmentation vs synthesis?** Transforming samples versus generating new ones.

17. Practice Tasks

Create constraint-based test data; compare correlations; apply SMOTE inside CV pipeline; train real-only versus augmented; detect invalid generated records.

18. Project Ideas

Project	Stack/data	Resume value
Fraud imbalance study	imbalanced-learn; credit-card fraud	Valid SMOTE/CV evaluation.
Synthetic tabular lab	SDV/CTGAN; Adult Income	Fidelity, utility, privacy analysis.
Defect augmentation	torchvision; MVTec	Vision robustness experiment.

19. Quick Revision

Synthetic data approximates useful real-data structure; validate utility on real held-out data. GAN aims for $p_g \approx p_{data}$. Trap: assuming realism equals privacy.

20. Final Cheat Sheet

Input/output: seed/real distribution → artificial samples. **Steps:** objective, generate, constrain, validate utility/privacy. **Metrics:** real-test performance, fidelity, privacy. **Pros:** scale/coverage. **Cons:** bias/artifacts/memorization. **Best:** augmentation and testing.